The Journal of Open Source Software

stimpack: a modular framework for precise multisensory stimulus generation in systems neuroscience

Minseung Choi [1¶], **Joshua B. Melander** [1], **Steven G. Herbst** [2], **Carl F. R. Wienecke** [1,3], **Heberto Mayorquin** [4], **Alex Y. Hao** [1], **Manze Zhang** [1], **Jacob C. Simon** [1], **David D. Au** [1], **Bella E. Brezovec** [1], **Stephen A. Baccus** [1], **Thomas R. Clandinin** [1,5*], **and Maxwell H. Turner** [1,6*¶]

1 Department of Neurobiology, Stanford University, USA **2** Department of Electrical Engineering, Stanford University, USA **3** Department of Neurobiology, Harvard Medical School, USA **4** CatalystNeuro, USA **5** Biohub, San Francisco, USA **6** Department of Biological Sciences, University at Albany, SUNY, USA ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

A central goal of neuroscience is to understand how animals engage with the natural world. Studying this experimentally requires the ability to present stimuli that capture the complexity of dynamic sensory inputs during animal behavior. But performing such experiments across labs, and interpreting the resulting data, can be challenging because doing so requires highly flexible and precisely defined stimuli that can be synchronized with other measurement tools, and can be reconstructed to facilitate data analysis.

To address these challenges, we introduce `stimpack`, a Python framework for open- and closed-loop stimulus delivery. `stimpack` is a hardware-agnostic stimulus rendering environment where a single protocol drives perspective-corrected displays across any number and arrangement of screens, sound playback, analog outputs (e.g. optogenetic stimulation, odor delivery, or reward), and closed-loop position updates. Stimuli and devices are **modules** addressed by name over a remote-procedure-call layer, and one protocol call can invoke several modules at once. Data are written through pluggable backends, including the neuroscience community standard Neurodata Without Borders (NWB) (Rübel et al., 2022; Teeters et al., 2015).

Everything specific to a laboratory or experimental setup (e.g. screen geometry, hardware drivers, stimulus definitions) is coded in a lab- or user-specific `labpack` outside the installed package, discovered at runtime from a configuration file. One protocol library runs across rigs with different screen geometries and hardware, allowing experiments to be replicated precisely across laboratories.

Statement of need

Existing software packages for parametric psychophysics make naturalistic, behavior-coupled stimuli difficult to implement, while packages designed for three-dimensional virtual environments make precise parameterization challenging. `stimpack` was designed for laboratories that position visual displays around a head-fixed or tethered animal and couple the outputs of these displays to animal behavior in closed loop. In this context, `stimpack` satisfies three important requirements. First, with `stimpack` the same visual stimulus can be shown across different presentation geometries, including setups with multiple displays or curved screens. A stimulus can be specified in terms of visual angles relative to the subject,

as in parametric psychophysics, or specified by Euclidean positions and sizes, as in a virtual environment. In either case, each display renders the stimulus at perspective-corrected angles (Figure 2) such that quantities measured against the stimulus, from receptive fields to speed tuning, emerge in the correct angular coordinates on any rig. Second, the same package can coordinate multisensory stimulus delivery, closed-loop control, and analog device control, allowing these devices to be synchronized precisely. Finally, `stimpack`'s files are valid NWB, so acquired data (e.g. physiological recordings, functional imaging, behavior tracking) can be added with `pynwb` or NeuroConv (Mayorquin et al., 2025), browsed with Neurosift (Magland et al., 2024), and archived on DANDI.

State of the field

We created `stimpack` to fill a gap by enabling flexible display geometries, native closed-loop control, modular device handling, and simple yet powerful stimulus design. Existing software packages for experiment control and visual stimulus delivery typically satisfy some, but not all, of these requirements.

Some visual display packages were built for **precise parametric control of a stimulus shown to a participant**. These packages make it easy to design visual stimuli, but can be difficult to extend to new screen geometries or incorporate closed-loop control based on behavioral feedback. For example, Psychtoolbox (Brainard, 1997; Pelli, 1997) and PsychoPy (Peirce et al., 2019) are designed around a display viewed roughly head-on, not around several surfaces at arbitrary orientations, and cannot readily be coupled to movement. A similar static-observer display model underlies tools for head-fixed physiology such as MonkeyLogic, PLDAPS, MWorks, and Rigbox (Bhagat et al., 2020; Eastman & Huk, 2012; Hwang et al., 2019; The MWorks Project, 2026), as well as in the Python packages Vision Egg (Straw, 2008) and QDSpy (Franke et al., 2019).

Other software packages have been purpose-built for use cases that involve **closed-loop control in a virtual environment**. These packages, while often delivering good performance, are often designed with specific hardware setups or requirements. For example, Stytra (Štih et al., 2019) and vxPy (Hladnik, 2026) have been used for a single preparation (larval zebrafish) while FreemoVR (Stowers et al., 2017) extended similar functionality to freely moving animals. Modular, custom LED arenas designed for work in fruit flies (Isaacson et al., 2022; Reiser & Dickinson, 2008) reach kilohertz binary frame rates with specific hardware requirements and a fixed geometry and resolution. Each of these packages allows for closed-loop control of parameterized stimuli, but does not directly support transferring experimental specifications to other rig geometries.

The existing packages that are most similar to `stimpack` can be extended to **diverse display types and geometries**. For example, ViRMEn (Aronov & Tank, 2014) and BonVision (Lopes et al., 2021) allow for curved displays, including conical and spherical screens. Bonsai (Lopes et al., 2015) handles closed-loop control and mixed devices, but trial structure and logging are left to the workflow's author, and rig calibration is embedded in the workflow itself. However, a `stimpack` protocol is a `diff`-able Python class whose parameters sweep across trials and can be stored in NWB. Finally, game engines like Unity or Unreal that have been used in neuroscience supply worlds rather than calibrated parametric stimuli, whether used directly (Jangraw et al., 2014) or wrapped for dome projection (Shapcott et al., 2025).

`stimpack` satisfies all these requirements, with a focus on a flexible architecture. Everything rig-specific is configured in a laboratory-specific `labpack` rather than inside experiment code, so **one protocol can be used across rigs and preparations**. To our knowledge, no existing package combines observer-centered perspective correction across an arbitrary number of flat screens and curved surfaces in one rig, closed-loop coupling to any locomotor tracker, non-visual outputs (sound, analog control) addressed from the same protocol, and trial-parameterized metadata saved in a community data standard by default.

The Journal of Open Source Software

Software design

One module per capability

In `stimpack`, an experiment runs as a **client**, which executes the protocol and writes metadata to the data file, and a **server**, which controls hardware devices, including display screens, as a set of modules. One command from the client can control several modules at once (Figure 1). Four modules, defined by the capability rather than a particular device, are included in `stimpack`: `visual`, which controls visual stimulus rendering and display; `audio`, which plays sound stimuli through a sound card; `locomotion`, which handles locomotion-related behavior data for closed-loop feedback; and `voltage_out`, which can send voltage waveforms to control devices (e.g. optogenetics, odor delivery, reward). This modular design choice allows a shared protocol library to run on rigs whose hardware may differ.

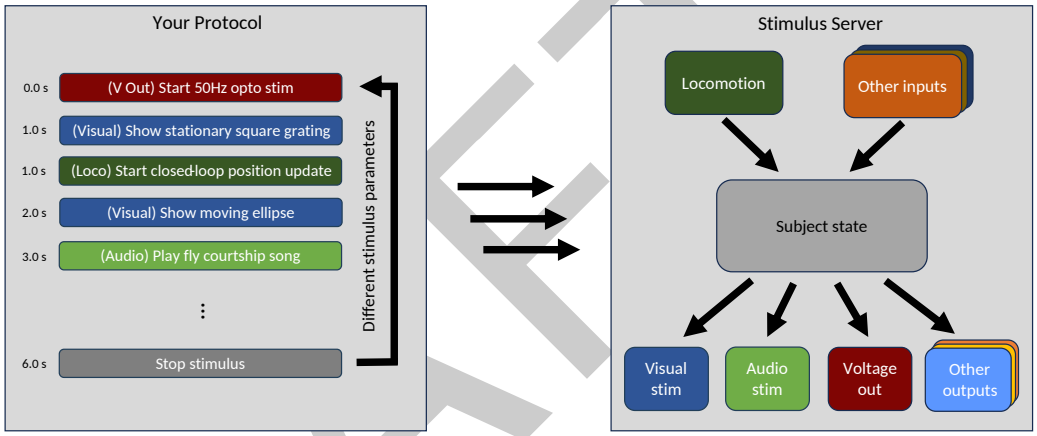

Figure 1: stimpack's architecture. A protocol names the module for each call, and the server routes it. Inputs update a subject state that outputs follow, such that the closed loop does not pass through the client. Stacked cards are extension points: a new capability is a new module, not a change to the core.

Visual module

Displays are defined in the server code, which includes a description of the screen geometry relative to the observer. Built-in stimuli range from parametric primitives to natural movies. Stimulus shapes can be painted with monochrome or RGB textures for stimuli like gratings, spatiotemporal white noise, or movie frames. Layering, masking, and aperturing of stimuli can be achieved by a combination of alpha blending and rendering surfaces with depth.

The `visual` module supports two display geometries that have separate rendering paths. **Flat screens** are described by their corner locations (in meters), and each stimulus is rendered per subscreen (one perspective view per flat surface) through a generalized off-axis perspective projection (Kooima, 2009), so an object subtends the angle it should from where the subject sits. **Curved screens** (e.g. a hemisphere or a cylinder) cannot be described by a flat projection frustum, so rendering happens in two steps. The scene is first rendered into a cube centered on the subject, capturing what the subject should see in every direction. The screen's mesh is drawn in display device (e.g. projector) coordinates, and each output pixel samples the cube along the direction of the screen point it shows (Figure 2).

The `visual` module supports most computer display devices. Temporal multiplexing packs up to three stimulus timepoints into the color channels of each output frame, so a 120 Hz video signal can drive a 360 Hz monochrome display, approaching the temporal regime of specialized LED hardware (Reiser & Dickinson, 2008). During stimulus presentation, a small square in the corner of the display inverts on every rendered frame to enable accurate synchronization with other recorded data and to enable detection of dropped frames (Figure 2d).

The Journal of Open Source Software

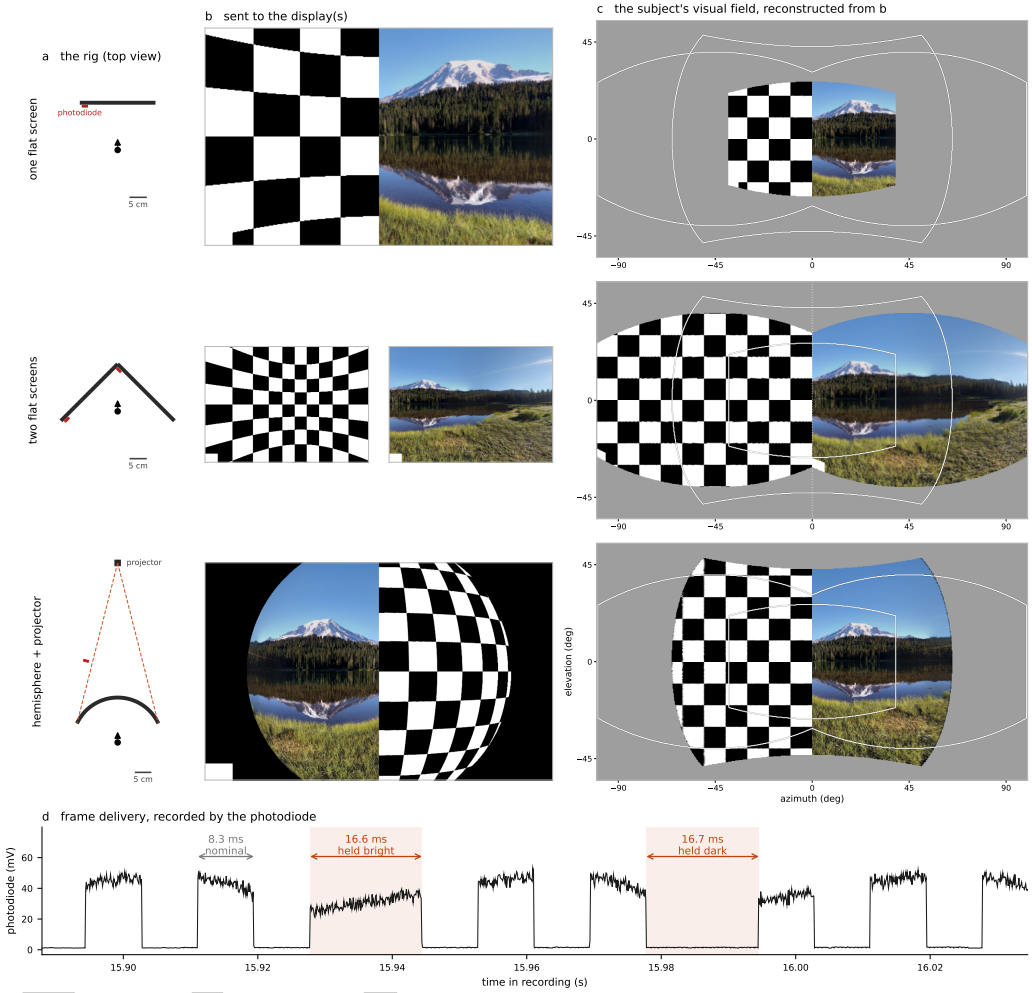

Figure 2: One stimulus specification, three display geometries, one visual experience. (a) Three rigs, to scale: one monitor; two monitors meeting ahead of the subject; a hemispherical screen lit by a projector. Red: the photodiode at each display's synchronization square. (b) The frames `stimpack` sends to the displays for one specified scene – a 10° checkerboard left of azimuth zero, and a 360° photograph right of it: `stimpack` warps the images according to the specification of each display. (c) The subject's visual field: the color seen in each direction is read off the delivered frame, at the display point visible in that direction. Grey marks directions with no display surface; faint outlines trace the coverages of the other two rigs for comparison across rows. (d) Frame delivery at the photodiode: nominal inversions every 8.3 ms (120 Hz), and two dropped frames, one held bright and one held dark.

Audio, locomotion, and voltage output modules

The `audio` module drives a sound card. A protocol addresses the sound card in the same way that the screens are addressed, naming a sound class whose waveform is rendered when the trial loads and played when the trial starts. For locomotion-based closed-loop control, lab-defined protocol code supplies a control function, which the server calls on every tracker update with the full subject state (position, orientation, and any laboratory-defined fields measured through `labpack`-defined trackers). The same interface lets a trial end on what the animal did, with the reason recorded. The voltage-output module supplies the dispatch; the calls themselves (steps, waveform streams, acquisition triggers) are defined and implemented by the `labpack` DAQ driver.

The Journal of Open Source Software

Protocols, data, and the interface

A protocol is a Python class in the `labpack` that declares the parameters of a run and of the stimulus. Named presets can be saved and loaded to standardize protocol parameter sets. Users can also run an ensemble, which is a saved queue of protocols. Data are written through pluggable backends: HDF5 by subject, series, and trial, with parameters at each level, or NWB (Rübel et al., 2022; Teeters et al., 2015), the community standard. `stimpack` includes a GUI that is a shell over these objects and identical on every rig (Figure 3).

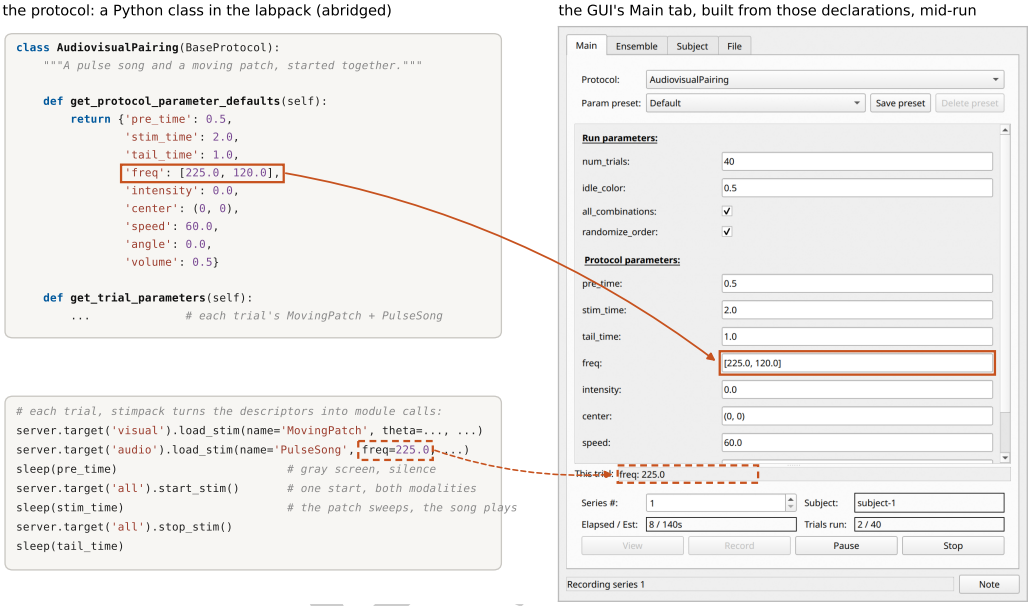

Figure 3: From protocol code to a running experiment. Left: the `AudiovisualPairing` protocol, abridged, and the module calls that `stimpack` makes from its descriptors on each trial – one `load_stim` per descriptor, routed by target; one start for both modalities; the timing declarations pacing the trial. Right: the GUI's Main tab mid-run, its parameter fields built from the class's own declarations. The list-valued `freq` (solid red box) sweeps across trials in randomized order, and "This trial" shows the current draw (dashed red boxes), here reaching the audio module.

Extensibility

`stimpack` is extensible at every layer. Stimulus classes load from the `labpack` at runtime and are addressed exactly as built-ins; data backends, trackers, and DAQ drivers are likewise `labpack` code. A new capability (e.g. a novel sensor or effector) is a three-line subclass of the same module base class the built-ins inherit, registered with the server.

Research impact statement

`stimpack` consolidates four earlier Clandinin lab packages: `flystim` (perspective-corrected rendering), `flyrpc` (client–server messaging), `visprotocol` (protocols, metadata, GUI), and `multistim` (auditory stimulation). None has been described in an archival publication, and every author of those packages is an author here. The subscreen geometry is based on that of `flystim`. The curved-screen path was checked against `flymax`, an earlier MATLAB-based hemisphere-projector display in the lab, and its measured photometry. Predecessor versions underlie published work on fly visual processing and behavior (Currier & Clandinin, 2025; Mano et al., 2023; Turner et al., 2022), the most recent of which names `flystim`, `visprotocol`, and `stimpack` itself. In addition, `stimpack` has been used to reconstruct spatiotemporal receptive fields in mouse retina and visual cortex (Au et al., 2026; Weddington et al., 2026), including

the example in Figure 4. The consolidation brought the modular architecture described above, improvements throughout each module, and three specific shifts. First, the viewer became a **subject with state**, which can be sent to every module, so one protocol can be used for a fly on a ball or a mouse on a treadmill. Second, **closed-loop control moved into the render loop**. Third, **laboratory-specific code left the core package**. Whereas `visprotocol` included lab-specific protocols and hardware drivers in its core, `stimpack` contains a generic core, and lab-specific code is housed in a `labpack`. `stimpack` is in use in ongoing *Drosophila* experiments in the Clandinin (Stanford), Turner (Albany), and Murthy (Princeton) laboratories, and in mouse work in the Baccus and Mitra laboratories (Stanford).

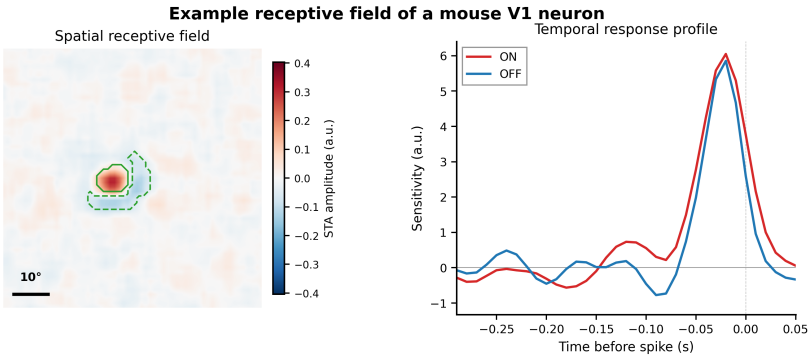

Figure 4: Spatiotemporal receptive field (STRF) of a mouse primary visual cortex neuron. The STRF of a cortical neuron recorded with a Neuropixels array was mapped with a white noise stimulus generated and rendered on two flat screens (similar to Figure 2, second row) with `stimpack`. The white noise stimulus had a 13° spatial correlation and a 33-ms temporal correlation. The eye was tracked to reconstruct the retinal image at a spatial scale of $< 1°$ (Au et al., 2026).

AI usage disclosure

All primary architectural design decisions and implementations were made by authors. Generative AI (Claude Opus 4.8, Opus 5, and Fable 5 from Anthropic) was used for parts of code generation and editing (multisample anti-aliasing, curved screen, audio module, experiment GUI elements), code merging (curved screen, NWB data), documentation, testing scaffolding, and manuscript revision. Authors thoroughly reviewed, modified, and validated all AI-generated content.

Acknowledgements

We thank the Clandinin, Turner, Baccus, Mitra, and Murthy Labs for testing across rigs and preparations. This work was supported by Stanford Graduate Fellowships (MC, SGH), a DoD NDSEG Fellowship (MC), a Hertz Fellowship (SGH), and NIH grants R00-EY032549, R01-EY022638, R01-EY022933, R01-EY025087, R01-EY034566 and P30-EY026877. TRC is a Biohub, San Francisco, Investigator.

References

Aronov, D., & Tank, D. W. (2014). Engagement of neural circuits underlying 2D spatial navigation in a rodent virtual reality system. *Neuron*, *84*(2), 442–456. https://doi.org/10.1016/j.neuron.2014.08.042

Au, D. D., Melander, J. B., Weddington, J. C., Faragalla, Y., Alaoui, Z., Liu, S., Xu, Q., & Baccus, S. A. (2026). Single-camera, calibration-free gaze estimation using corneal

reflections. *bioRxiv*. https://doi.org/10.64898/2026.06.08.731021

Bhagat, J., Wells, M. J., Harris, K. D., Carandini, M., & Burgess, C. P. (2020). Rigbox: An open-source toolbox for probing neurons and behavior. *eNeuro*, *7*(4), ENEURO.0406–19.2020. https://doi.org/10.1523/ENEURO.0406-19.2020

Brainard, D. H. (1997). The Psychophysics Toolbox. *Spatial Vision*, *10*(4), 433–436. https://doi.org/10.1163/156856897X00357

Currier, T. A., & Clandinin, T. R. (2025). Infrequent strong connections constrain connectomic predictions of neuronal function. *Cell*, *188*(16), 4366–4381.e14. https://doi.org/10.1016/j.cell.2025.05.007

Eastman, K. M., & Huk, A. C. (2012). PLDAPS: A hardware architecture and software toolbox for neurophysiology requiring complex visual stimuli and online behavioral control. *Frontiers in Neuroinformatics*, *6*, 1. https://doi.org/10.3389/fninf.2012.00001

Franke, K., Maia Chagas, A., Zhao, Z., Zimmermann, M. J. Y., Bartel, P., Qiu, Y., Szatko, K. P., Baden, T., & Euler, T. (2019). An arbitrary-spectrum spatial visual stimulator for vision research. *eLife*, *8*, e48779. https://doi.org/10.7554/eLife.48779

Hladnik, T. (2026). *vxPy: Multiprocess-based software for vision experiments in Python*. https://github.com/thladnik/vxPy

Hwang, J., Mitz, A. R., & Murray, E. A. (2019). NIMH MonkeyLogic: Behavioral control and data acquisition in MATLAB. *Journal of Neuroscience Methods*, *323*, 13–21. https://doi.org/10.1016/j.jneumeth.2019.05.002

Isaacson, M., Ferguson, L., Loesche, F., Ganguly, I., Chen, J., Chiu, A., Liu, J., Dickson, W., & Reiser, M. (2022). A high-speed, modular display system for diverse neuroscience applications. *bioRxiv*. https://doi.org/10.1101/2022.08.02.502550

Jangraw, D. C., Johri, A., Gribetz, M., & Sajda, P. (2014). NEDE: An open-source scripting suite for developing experiments in 3D virtual environments. *Journal of Neuroscience Methods*, *235*, 245–251. https://doi.org/10.1016/j.jneumeth.2014.06.033

Kooima, R. (2009). *Generalized perspective projection*. Louisiana State University.

Lopes, G., Bonacchi, N., Frazão, J., Neto, J. P., Atallah, B. V., Soares, S., Moreira, L., Matias, S., Itskov, P. M., Correia, P. A., Medina, R. E., Calcaterra, L., Dreosti, E., Paton, J. J., & Kampff, A. R. (2015). Bonsai: An event-based framework for processing and controlling data streams. *Frontiers in Neuroinformatics*, *9*, 7. https://doi.org/10.3389/fninf.2015.00007

Lopes, G., Farrell, K., Horrocks, E. A. B., Lee, C.-Y., Morimoto, M. M., Muzzu, T., Papanikolaou, A., Rodrigues, F. R., Wheatcroft, T., Zucca, S., Solomon, S. G., & Saleem, A. B. (2021). Creating and controlling visual environments using BonVision. *eLife*, *10*, e65541. https://doi.org/10.7554/eLife.65541

Magland, J., Soules, J., Baker, C., & Dichter, B. (2024). Neurosift: DANDI exploration and NWB visualization in the browser. *Journal of Open Source Software*, *9*(97), 6590. https://doi.org/10.21105/joss.06590

Mano, O., Choi, M., Tanaka, R., Creamer, M. S., Matos, N. C. B., Shomar, J. W., Badwan, B. A., Clandinin, T. R., & Clark, D. A. (2023). Long-timescale anti-directional rotation in Drosophila optomotor behavior. *eLife*, *12*, e86076. https://doi.org/10.7554/eLife.86076

Mayorquin, H., Baker, C., Adkisson-Floro, P., Weigl, S., Trapani, A., Tauffer, L., Rübel, O., & Dichter, B. (2025). NeuroConv: Streamlining neurophysiology data conversion to the NWB standard. *Proceedings of the 24th Python in Science Conference*. https://doi.org/10.25080/cehj4257

Peirce, J., Gray, J. R., Simpson, S., MacAskill, M., Höchenberger, R., Sogo, H., Kastman, E., & Lindeløv, J. K. (2019). PsychoPy2: Experiments in behavior made easy. *Behavior*

Research Methods, *51*(1), 195–203. https://doi.org/10.3758/s13428-018-01193-y

Pelli, D. G. (1997). The VideoToolbox software for visual psychophysics: Transforming numbers into movies. *Spatial Vision*, *10*(4), 437–442. https://doi.org/10.1163/156856897X00366

Reiser, M. B., & Dickinson, M. H. (2008). A modular display system for insect behavioral neuroscience. *Journal of Neuroscience Methods*, *167*(2), 127–139. https://doi.org/10.1016/j.jneumeth.2007.07.019

Rübel, O., Tritt, A., Ly, R., Dichter, B. K., Ghosh, S., Niu, L., Baker, P., Soltesz, I., Ng, L., Svoboda, K., Frank, L., & Bouchard, K. E. (2022). The Neurodata Without Borders ecosystem for neurophysiological data science. *eLife*, *11*, e78362. https://doi.org/10.7554/eLife.78362

Shapcott, K. A., Weigand, M., Glukhova, M., Havenith, M. N., & Schölvinck, M. L. (2025). DomeVR: Immersive virtual reality for primates and rodents. *PLOS ONE*, *20*(1), e0308848. https://doi.org/10.1371/journal.pone.0308848

Štih, V., Petrucco, L., Kist, A. M., & Portugues, R. (2019). Stytra: An open-source, integrated system for stimulation, tracking and closed-loop behavioral experiments. *PLOS Computational Biology*, *15*(4), e1006699. https://doi.org/10.1371/journal.pcbi.1006699

Stowers, J. R., Hofbauer, M., Bastien, R., Griessner, J., Higgins, P., Farooqui, S., Fischer, R. M., Nowikovsky, K., Haubensak, W., Couzin, I. D., Tessmar-Raible, K., & Straw, A. D. (2017). Virtual reality for freely moving animals. *Nature Methods*, *14*(10), 995–1002. https://doi.org/10.1038/nmeth.4399

Straw, A. D. (2008). Vision Egg: An open-source library for realtime visual stimulus generation. *Frontiers in Neuroinformatics*, *2*, 4. https://doi.org/10.3389/neuro.11.004.2008

Teeters, J. L., Godfrey, K., Young, R., Dang, C., Friedsam, C., Wark, B., Asari, H., Peron, S., Li, N., Peyrache, A., Denisov, G., Siegle, J. H., Olsen, S. R., Martin, C., Chun, M., Tripathy, S., Blanche, T. J., Harris, K., Buzsáki, G., … Sommer, F. T. (2015). Neurodata without borders: Creating a common data format for neurophysiology. *Neuron*, *88*(4), 629–634. https://doi.org/10.1016/j.neuron.2015.10.025

The MWorks Project. (2026). *MWorks: A framework for conducting realtime neurophysiology and psychophysics experiments*. https://mworks.github.io/

Turner, M. H., Krieger, A., Pang, M. M., & Clandinin, T. R. (2022). Visual and motor signatures of locomotion dynamically shape a population code for feature detection in Drosophila. *eLife*, *11*, e82587. https://doi.org/10.7554/eLife.82587

Weddington, J. C., Au, D. D., Melander, J. B., Faragalla, Y., Alaoui, Z., & Baccus, S. A. (2026). Parallel construction of object motion from retina to cortex. *bioRxiv*. https://doi.org/10.64898/2026.07.27.740872